

TUP



Gestión de Desarrollo de Software

Unidad I

Introducción a la gestión de proyectos de software





Las 4 P's

La ingeniería de sistemas propone un marco de trabajo fundamental para lograr buenos resultados en el **proceso** de desarrollo (construcción) de un **producto/solución** en el contexto de un **proyecto** de software. Nos basaremos en el libro de *Roger Pressman, Ingeniería de Software: un enfoque práctico*. La administración efectiva de un proyecto de software se enfoca en las cuatro P: personas, producto, proceso y proyecto.

- **Personas**
- **Producto**
- **Proceso**
- **Proyecto**





Personas

Todo proyecto de software está poblado de participantes, quienes pueden ubicarse en alguna de las siguientes áreas: Gerentes Ejecutivos, Gerentes de proyecto, Profesionales, Clientes, Usuarios finales.

Líder de Proyecto

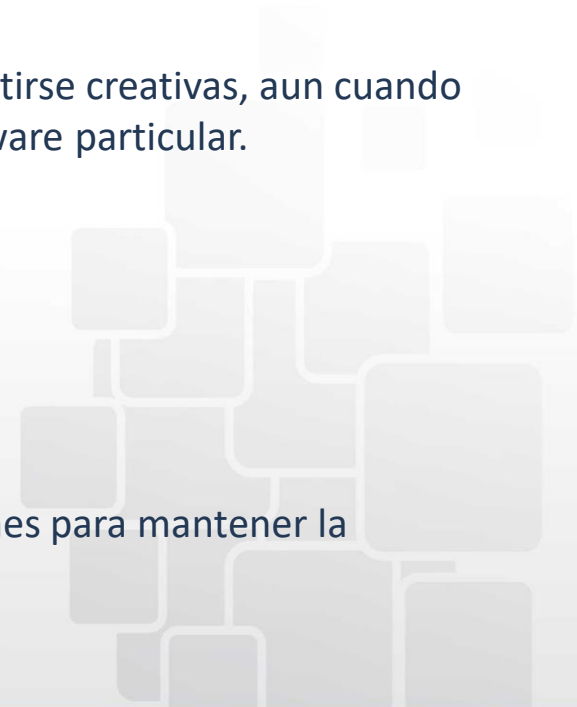
Un buen líder/gerente de proyecto debería cumplir con las siguientes características o capacidades:

- **Motivación.** Habilidad para alentar al personal técnico a producir a su máxima capacidad.
- **Organización.** Habilidad para moldear los procesos existentes (o inventar nuevos) que permitirán que el concepto inicial se traduzca en un producto final.
- **Ideas o innovación.** Habilidad para alentar a las personas a crear y sentirse creativas, aun cuando deban trabajar dentro de fronteras establecidas para un producto/software particular.

Equipo

Para lograr un equipo de alto rendimiento se requiere que:

- Los miembros del equipo tengan confianza entre sí.
- La distribución de habilidades sea la adecuada para el problema.
- Si es necesario, quizás tengan que excluirse del equipo a los inconformes para mantener la cohesión del equipo.





Personas

“Tendemos a usar la palabra equipo con mucha holgura en el mundo empresarial, y llamamos así a cualquier grupo de personas asignadas para trabajar juntas. Pero muchos de estos grupos simplemente no parecen equipos.”

“Un equipo cuajado es un grupo de personas tan fuertemente unido que el todo es mayor que la suma de las partes.”

Toxicidad de Equipo

Pueden mencionarse cinco factores que “fomentan un ambiente de equipo potencialmente tóxico”:

- 1) una atmósfera de trabajo frenético,
- 2) alta frustración, que causa fricción entre los miembros del equipo,
- 3) un proceso de software “fragmentado o pobremente coordinado”,
- 4) una definición poco clara de los roles en el equipo de software y
- 5) continua y repetida exposición al fracaso.





Producto

¿Qué es?

- Es el software.
- Se sustenta en otros productos generados por los Ingenieros en Sistemas, los arquitectos de software, los diseñadores y los analistas.
- Como producto podemos afirmar que luego de su puesta en marcha necesita que se lo mantenga.

¿Quién lo hace?

- Ingenieros en Sistemas / Ingenieros de Software
- Arquitectos de Software
- Diseñadores
- Analistas
- Programadores

¿Cuáles son los pasos para obtenerlo?

- Los pasos estarán definidos por la **metodología** que se elija para llevar adelante el proyecto.
- Pueden ser: *lineales, incrementales, iterativas o evolutivas, y ágiles.*





Producto

¿Qué se obtiene como producto final?

Es decir, del punto de vista del que construye el producto de software, ¿de que se compone?

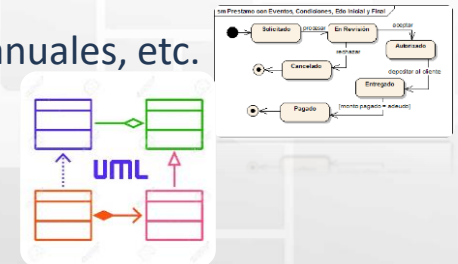
- Programas, archivos ejecutables, código fuente.



- Diseño lógico de datos.



- Documentación: especificaciones, modelos, diagramas, instructivos, manuales, etc.





Producto

¿Qué ocurre con el producto final?

Retomando el concepto de software como producto, debemos comprender que el software tiene casi las mismas características que otros productos que resultan de otras actividades ingenieriles. En este sentido, al igual que el motor de un auto se desgasta con el tiempo, el software sufre, no un deterioro físico, sino que se vuelve obsoleto. Es decir, ya no logra satisfacer nuevos requerimientos o soportar cambios sin perder calidad. Comparemos entonces como es el comportamiento del **hardware** y el **software** en el tiempo para entenderlo mejor.



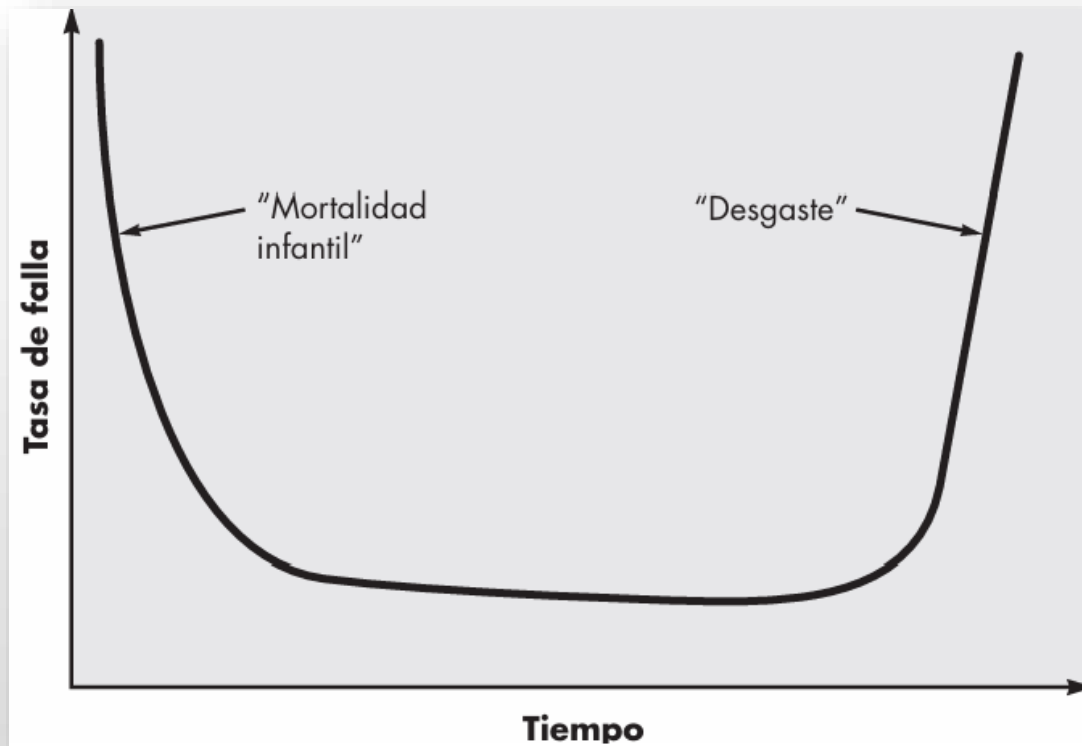


Producto

¿Qué ocurre con el producto final?

La curva indica que el hardware presenta una tasa de fallas relativamente elevada en una etapa temprana de su vida (fallas que con frecuencia son atribuibles a defectos de diseño o manufactura); los defectos se corrigen y la tasa de fallas se abate a un nivel estable durante cierto tiempo. No obstante, conforme pasa el tiempo, la tasa de fallas aumenta de nuevo a medida que los componentes del hardware resienten los efectos acumulativos de suciedad, vibración, abuso, temperaturas extremas y muchos otros inconvenientes ambientales. En pocas palabras, *el hardware comienza a desgastarse*.

Curva de fallas del Hardware



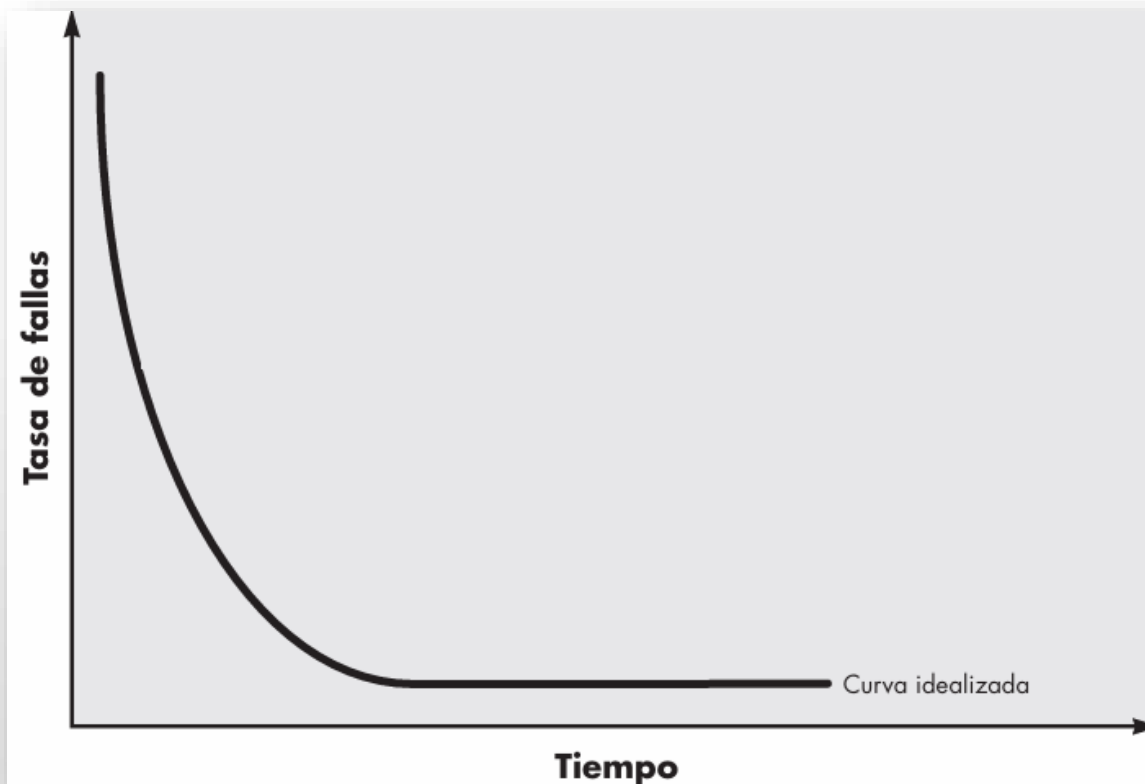


Producto

¿Qué ocurre con el producto final?

El software no es susceptible a los problemas ambientales que hacen que el hardware se desgaste. Por tanto, en teoría, la curva de la tasa de fallas adopta la forma de la “curva idealizada” que se aprecia en la figura. Pareciera que los defectos ocultos ocasionarán tasas elevadas de fallas al comienzo de la vida de un programa. Sin embargo, éstas se corrigen y la curva se aplanan, como se indica. La curva idealizada es una gran simplificación de los modelos reales de las fallas del software.

Curva ideal de fallas del Software



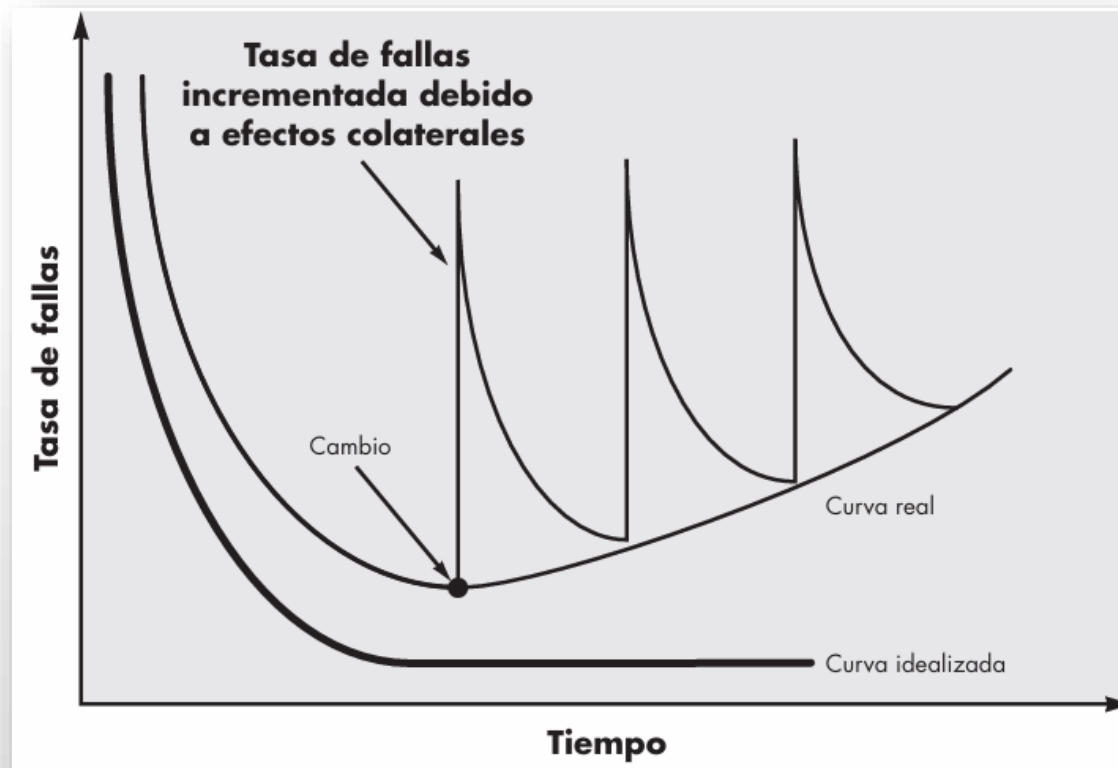


Producto

¿Qué ocurre con el producto final?

La implicación está clara: el software no se desgasta, *¡pero sí se deteriora!* Durante su vida, el software sufrirá cambios. Es probable que cuando éstos se realicen, se introduzcan errores que ocasionen que la curva de tasa de fallas tenga aumentos súbitos, como se ilustra en la “curva real”. Antes de que la curva vuelva a su tasa de fallas original de estado estable, surge la solicitud de otro cambio que hace que la curva se dispare otra vez. Poco a poco, el nivel mínimo de la tasa de fallas comienza a aumentar: el software se está deteriorando como consecuencia del cambio.

Curva real de fallas del Software





Proceso

¿Qué es un Proceso de Software?

Se define proceso del software como una estructura para las actividades, acciones y tareas que se requieren a fin de construir software de alta calidad.

- Un proceso de software es el **marco de trabajo** que define el enfoque que se toma cuando el software es tratado por la ingeniería.
- Se encuentra constituido por un conjunto de actividades estructurales, acciones y tareas que se requieren para desarrollar software de alta calidad.
- Una **actividad** busca lograr un objetivo amplio (por ejemplo, comunicación con los participantes). Una **acción** es un conjunto de tareas que producen un producto importante del trabajo. Una **tarea** se centra en un objetivo pequeño pero bien definido (por ejemplo, realizar una prueba unitaria).

Proceso del software

Estructura del proceso

Actividades

actividad estructural # 1

acción de ingeniería de software # 1.1

Conjuntos
de tareas
⋮

tareas del trabajo
productos del trabajo
puntos de aseguramiento de la calidad
puntos de referencia del proyecto

acción de ingeniería de software # 1.k

Conjuntos
de tareas

tareas del trabajo
productos del trabajo
puntos de aseguramiento de la calidad
puntos de referencia del proyecto

⋮

actividad estructural # n

acción de ingeniería de software # n.1

Conjuntos
de tareas
⋮

tareas del trabajo
productos del trabajo
puntos de aseguramiento de la calidad
puntos de referencia del proyecto

acción de ingeniería de software # n.m

Conjuntos
de tareas

tareas del trabajo
productos del trabajo
puntos de aseguramiento de la calidad
puntos de referencia del proyecto



Proceso

Marco de Trabajo Genérico

Cada actividad estructural está formada por un conjunto de acciones de ingeniería de software y cada una de éstas se encuentra definida por un conjunto de tareas que identifica: las tareas del trabajo que deben realizarse, los productos del trabajo que se producirán, los puntos de aseguramiento de la calidad que se requieren y los puntos de referencia que se utilizarán para evaluar el avance.

Una estructura general para la ingeniería de software define cinco actividades estructurales: **comunicación, planeación, modelado, construcción y despliegue**. Además, a lo largo de todo el proceso se aplica un conjunto de actividades sombrilla o custodiales: seguimiento y control del proyecto, administración de riesgos, aseguramiento de la calidad, administración de la configuración, revisiones técnicas, entre otras.





Proceso

¿Cómo se organizan estas actividades?

Estas actividades se organizan de diferente manera según el modelo de proceso de desarrollo a utilizar:

- Lineal
- Incremental
- Iterativo o Evolutivo
- Iterativo e Incremental
- Ágiles

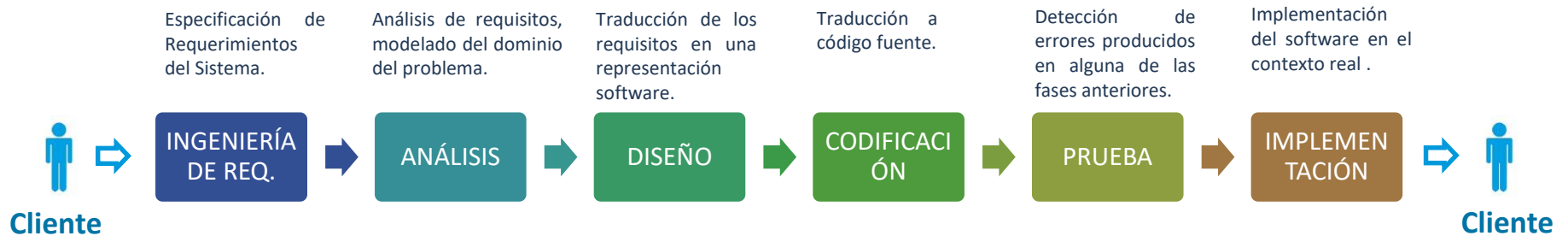
Cabe destacar que, ***cualquiera de estos modelos puede ser implementado con éxito***. Ello depende del ámbito de aplicación, del tamaño del dominio del problema (complejidad), y la incertidumbre.

LINEAL	INCREMENTAL	ITERATIVO o EVOLUTIVO	ITERATIVO E INCREMENTAL	AGILES
Cascada (waterfall)	Incremental	Prototipos	Proceso Unificado	Scrum
	Desarrollo Rápido de Aplicaciones (DRA)	Espiral	Iconix	Kanban
				XP



Proceso

Modelo Cascada (Waterfall)

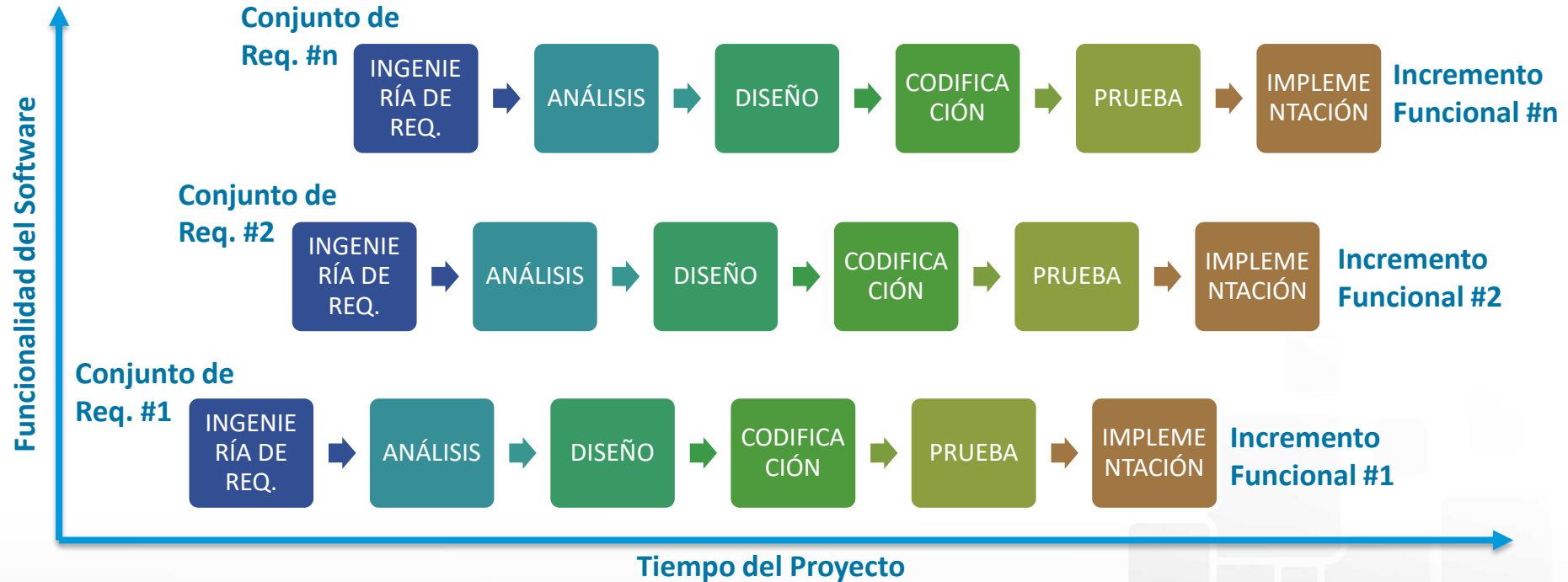


- Sugiere un enfoque sistemático y secuencial para el desarrollo del software basado en la especificación del cliente.
- Escasa participación del cliente a lo largo del proceso (interacción al comienzo y al final).
- Requiere una especificación de requerimientos completa al comienzo del proyecto.
- Detección tardía de problemas, en consecuencia, un error o falla requiere un esfuerzo importante para su corrección en etapas más avanzadas del proceso.
- El cliente puede ver una versión ejecutable del software recién al final del proceso.
- Hay división del trabajo.
- No hay trabajo en paralelo.
- Este modelo describe una serie de pasos genéricos que son aplicables a cualquier otro modelo. Por ello se lo suele denominar “*proceso madre*”.



Proceso

Modelo Incremental

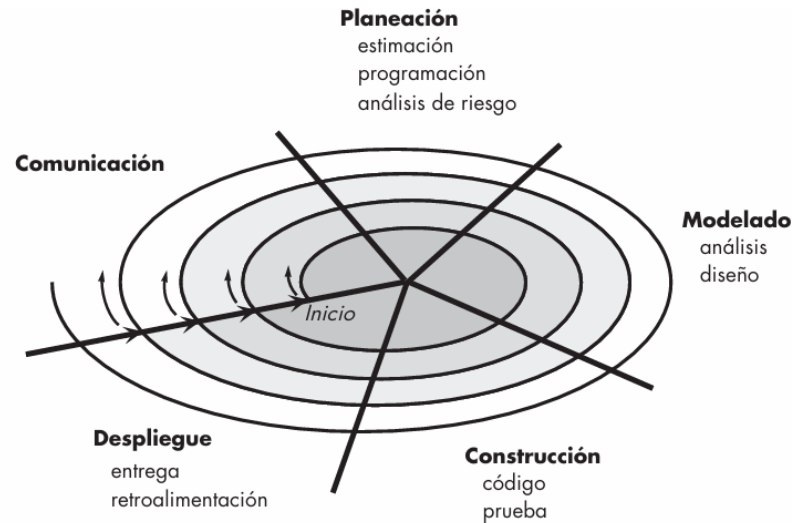


- Secuencial por incrementos de funcionalidad (se selecciona un conjunto de requerimientos en cada incremento y se produce una funcionalidad diferente al incremento anterior).
- No hay refinamiento de la funcionalidad.
- Escasa participación del cliente (interacción al comienzo y al final de cada incremento).
- El cliente puede ver una versión ejecutable del software al final de cada incremento.
- Hay división del trabajo.
- No hay trabajo en paralelo.



Proceso

Modelo Espiral



- Es un modelo de proceso evolutivo o iterativo.
- Hay refinamiento de la funcionalidad basada en la retroalimentación, hasta llegar a la versión completa deseada por el cliente.
- Se incorpora el concepto de riesgo y se lo gestiona.
- Enfoque cíclico para el crecimiento del grado de definición e implementación, mientras disminuye su grado de riesgo.
- Conjunto de puntos de fijación para asegurar el compromiso del usuario con soluciones factibles y satisfactorias. Uso de prototipos.
- Hay división del trabajo.
- No hay trabajo en paralelo.
- Orientado a proyectos donde el aprendizaje ocurre durante el desarrollo.



Proceso

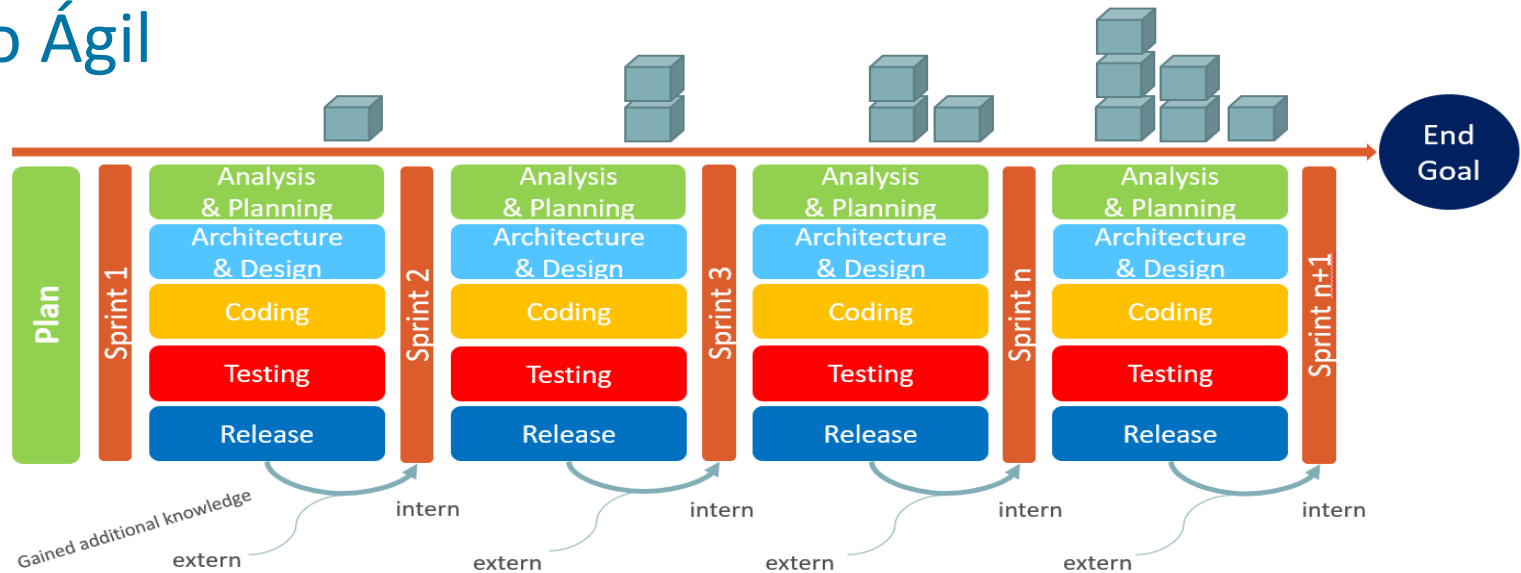
Proceso Unificado

- Desarrollado por Jacobson, Rumbaugh y Booch, UML brindaba la tecnología necesaria para apoyar la práctica de la ingeniería de software orientada a objetos, pero no proporcionaba la estructura del proceso que guiara a los equipos del proyecto cuando aplican la tecnología (UML).
- Es un modelo de proceso iterativo e incremental para la ingeniería de software orientada a objetos, mediante la utilización de UML (Unified Modeling Language).
- Hay refinamiento e integración de la funcionalidad anterior hasta llegar a la versión completa deseada por el cliente con cada incremento. Pero en cada incremento se obtiene una versión ejecutable que representa el grado de avance.
- Dirigido por Casos de Uso (representan los requisitos funcionales y guían el diseño, la prueba y la implementación).
- Centrado en la Arquitectura. La arquitectura no es solo un diseño, sino el **pilar que guía y controla el desarrollo**.
- Hay división del trabajo.
- La arquitectura y los casos de uso evolucionan en paralelo.
- El desarrollo se organiza en una serie de mini proyectos de duración fija, llamados iteraciones (2 a 6 semanas). El resultado de cada uno es un sistema que puede ser integrado, probado y ejecutado.



Proceso

Proceso Ágil



- Proceso iterativo e incremental
- Iteraciones cortas (sprints) de 1 a 4 semanas
- Colaboración continua con el cliente
- Software funcional por sobre documentación exhaustiva
- Responder al cambio en lugar de seguir un plan
- Equipos multifuncionales y autoorganizados
- Entrega rápida de software utilizable
- Alta adaptabilidad a los requisitos cambiantes
- Retroalimentación y mejora continuas
- Enfoque en el valor para el cliente



Proceso

Proceso Ágil - Manifiesto

Individuos e interacciones

- por sobre procesos y herramientas

Software funcionando

- por sobre documentación extensiva

Colaboración con el cliente

- por sobre negociaciones contractuales

Responder al cambio

- por sobre seguir un plan

Si bien los elementos de la derecha tienen valor, los elementos de la izquierda son más valiosos.

<https://agilemanifesto.org/>



Proceso

Resumen

- **Predictivos:** se tiene conocimiento total (o prácticamente completo) de los requerimientos. Todos los requisitos se conocen desde el inicio, por lo cual, es posible realizar una planificación completa (o prácticamente completa) y así definir costos, tiempos, etc. desde el arranque. Se realiza una única entrega al final del ciclo. Ejemplo: construcción de casa.
- **Iterativos:** no se tiene un conocimiento total de los requerimientos, hay cierta incertidumbre. A medida que se avanza en el desarrollo, con cada iteración se realiza una entrega evolucionada del producto. Cada entrega permite obtener una retroalimentación y realizar correcciones/mejoras a partir de la misma para luego incluirlas en la próxima iteración. Así, con cada iteración, se va logrando el perfeccionamiento/ampliación del producto. La entrega en cada iteración no necesariamente será una entrega definitiva de las funcionalidades desarrolladas hasta el momento. El objetivo es una única entrega al final del ciclo. Ejemplo: construcción de un robot.
- **Incrementales:** no se tiene un conocimiento total de los requerimientos, hay cierta incertidumbre. A medida que se avanza en el desarrollo, con cada iteración se realiza una entrega funcional real que se va adicionando al producto incrementando las funcionalidades del mismo y quedando así a disposición del cliente para que el mismo comience a utilizarla.
- **Ágiles:** no se tiene un conocimiento total de los requerimientos, demasiada incertidumbre. A medida que se avanza en el desarrollo, con cada iteración se realiza una entrega funcional real pero a la vez aporta una retroalimentación para la próxima iteración. Combina iterativo e incremental. Debe respetar los valores del manifiesto ágil.



Proceso

Resumen

- Existen diversos modelos de ciclo de vida de desarrollo de software (cascada, iterativo-incremental, ágil) y ofrecen diferentes enfoques para organizar las actividades de desarrollo, cada uno con distintas ventajas para contextos de proyectos específicos.
- El contexto importa: la selección del modelo de proceso apropiado requiere una cuidadosa consideración de las características del proyecto, las capacidades del equipo, la cultura organizacional y los requisitos de las partes interesadas.

Aplicación Educativa	Sistema Bancario	Sist. Comercial con Login	Control de Acceso Escolar
Considere factores como la importancia de los comentarios de los usuarios, la evolución del contenido educativo y la necesidad de actualizaciones frecuentes basadas en la participación de los estudiantes.	Evalúe los requisitos de cumplimiento de seguridad, la precisión de las transacciones, la integridad de los datos y las consecuencias de los errores en las operaciones financieras.	Considere la incertidumbre del mercado, el potencial de cambio rápido, los recursos limitados y la necesidad de validar rápidamente los supuestos comerciales.	Analice los requisitos de seguridad, la integración con los sistemas escolares existentes, los flujos de trabajo claramente definidos y las necesidades de cumplimiento normativo.



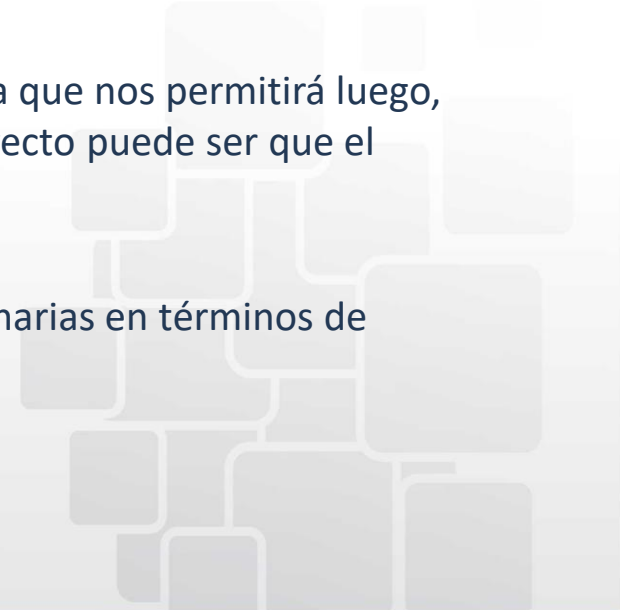
Proyecto

Un proyecto de software es ***un esfuerzo temporal emprendido para crear un producto***, servicio o resultado de software único. Todo proyecto se caracteriza por:

- Alcance y entregables definidos, objetivos específicos y medibles que definen los criterios de éxito y guían la toma de decisiones durante la ejecución.
- Cronograma fijo con fechas de inicio y fin, la naturaleza temporal impulsa la urgencia y el enfoque.
- Recursos asignados (humanos, financieros, técnicos), opera dentro de las limitaciones de presupuesto, personal, equipo y tiempo, lo que requiere una asignación y gestión cuidadosa.
- Expectativas de calidad y criterios de aceptación.
- Gestión activa, los proyectos requieren una planificación, un seguimiento y un control continuo para sortear las complejidades y garantizar una entrega exitosa

Es importante decir que debemos registrar antecedentes, la historia, ya que nos permitirá luego, realizar mejores estimaciones de costos y tiempos. En el siguiente proyecto puede ser que el producto cambie o tenga características similares.

Los proyectos difieren fundamentalmente de las tareas operativas rutinarias en términos de planificación, ejecución y gestión.





Proyecto

Proyectos vs Tareas Rutinarias

Características	Proyecto	Tarea Rutinaria
Duración	Temporal, con final definido	Continua, repetitiva
Singularidad	Crea resultados únicos	Produce resultados similares
Planificación	Requiere una planificación exhaustiva	Sigue procedimientos establecidos
Incertidumbre	Mayor riesgo, cambios frecuentes	Proceso predecible y estable
Recursos	Asignación variable	Necesidades de recursos consistentes
Gestión	Enfoque de gestión de proyectos	Gestión operativa





Proyecto

Ejemplificando:

Proyectos	Tareas Rutinarias
• Desarrollar una aplicación móvil para un festival local • Implementar un nuevo sistema de gestión de relaciones con los clientes • Crear una plataforma de aprendizaje electrónico personalizada para una universidad • Rediseñar el sitio web de una organización • Crear una solución de gestión de inventario para un minorista	• Facturación mensual de clientes • Copias de seguridad diarias del sistema • Sesiones semanales de corrección de errores • Mantenimiento regular del sistema • Atención al cliente continua



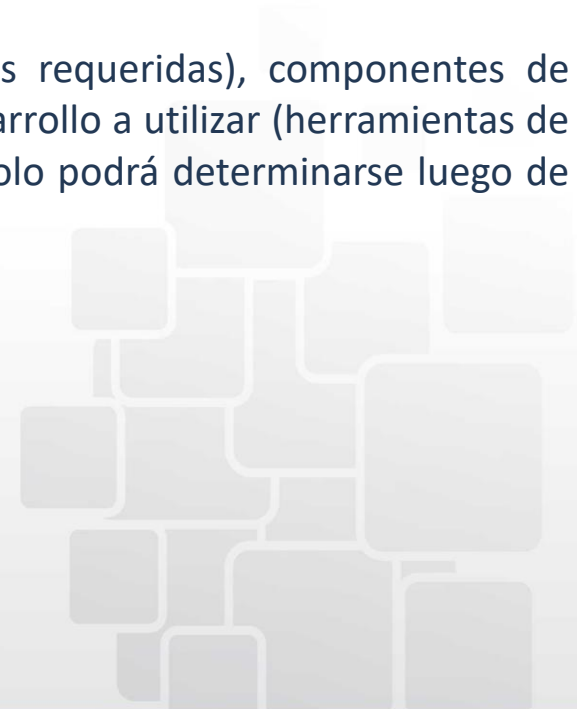
Planificación

Planificación de un Proyecto de Software

El objetivo de la planificación del proyecto de software es proporcionar un marco conceptual que permita al gerente hacer estimaciones razonables de recursos, costo y calendario. Además, las estimaciones deben intentar definir los escenarios de mejor caso y peor caso, de modo que los resultados del proyecto puedan acotarse.

La planificación de un proyecto involucra un conjunto de tareas:

- Establecer el ámbito del proyecto y factibilidad del mismo
- Analizar los Riesgos
- Definir los recursos requeridos: personal (roles y especialidades requeridas), componentes de software reutilizables (propios, de terceros, etc.) y entorno de desarrollo a utilizar (herramientas de hardware y software). Obviamente, la cantidad de cada recurso solo podrá determinarse luego de un estimación del esfuerzo requerido.
- Estimar costos y esfuerzo
- Desarrollar el calendario del proyecto





Planificación

Planificación de un Proyecto de Software

Para administrar un proyecto de software exitoso, se debe tener presente todo aquello que pueda salir mal.

Algunas señales que indican que un proyecto de sistemas de información está en peligro:

- El personal del software no entiende las necesidades del cliente.
- El ámbito del producto está pobremente definido.
- Los cambios se gestionan pobremente.
- Las fechas límite son irreales.
- Los usuarios son resistentes.
- El equipo del proyecto carece de personal con habilidades adecuadas.





Planificación

Caso 1: Desarrollo **planificado**

Un equipo sigue una metodología estructurada con requisitos claros, documentación y revisiones periódicas:

- Análisis detallado de requisitos
- Fase de diseño integral
- Revisiones de código programadas
- Estrategia de prueba sistemática
- Gestión formal de cambios

Caso 2: Desarrollo **improvisado**

Un equipo trabaja sin procesos formales, confiando en la experiencia individual:

- Recopilación verbal de requisitos
- Documentación de diseño mínima
- Prácticas de codificación ad hoc
- Pruebas dirigidas por desarrolladores
- Resolución reactiva de problemas

Pregunta de debate: ¿Qué enfoque tiene más probabilidades de tener éxito y por qué? Considere factores como la calidad, el cumplimiento del cronograma, el presupuesto y la satisfacción del equipo.



Administración del Riesgo

Riesgo de Software

Un proyecto de software es una empresa compleja, en consecuencia, un sinnúmero de situaciones pueden salir mal. Un riesgo es un problema potencial (latente).

Los riesgos siempre involucran dos características: **incertidumbre** (el riesgo puede o no ocurrir; es decir, no hay riesgos 100 por ciento probables) y **pérdida** (si el riesgo se vuelve una realidad, ocurrirán consecuencias o pérdidas no deseadas).

Cuando se analizan los riesgos es importante cuantificar el nivel de incertidumbre y el grado de pérdida asociado con cada riesgo.

Con respecto a los riesgos se pueden tomar dos estrategias: **reactivas** o **proactivas**.

Las estrategias **reactivas** de riesgo se basan en no preocuparse por los problemas hasta que sucedan (“No te preocupes, ¡pensaré en algo!”).

Una estrategia considerablemente más inteligente para la administración del riesgo es ser proactivo.

Una estrategia **proactiva** identifica los riesgos potenciales, valoran su probabilidad e impacto y los clasifica por importancia. Luego, el equipo de software establece un plan para gestionar el riesgo. El objetivo principal es evitarlo, pero, dado que no todos los riesgos son evitables, el equipo trabaja para desarrollar un plan de contingencia que le permitirá responder de forma controlada y efectiva.



Administración del Riesgo

Riesgo de Software

Una estrategia efectiva debe considerar los siguientes temas: 1-evitar el riesgo, 2-monitorear el riesgo, 3-manejar el riesgo y planificar la contingencia.

Pueden identificarse 3 categorías de riesgos: **Riesgos de Proyecto** (presupuesto, personal, recursos, etc.), **Riesgos Técnicos** (diseño, implementación, mantenimiento, etc.) y **Riesgos Empresariales** (producto no requerido por el mercado, el producto ya no encaja con la estrategia de la empresa, etc.). También es posible categorizar los riesgos en: riesgos conocidos (pueden descubrirse a partir de la evaluación del plan del proyecto o de otras fuentes de información: fecha de entrega irreal, falta de requisitos documentados, etc.), predecibles (pueden conocerse a partir de la experiencia sobre proyectos anteriores: rotación de personal, pobre comunicación con el cliente, etc.) e impredecibles (extremadamente difíciles de identificar por adelantado).

Una **tabla de riesgos** proporciona una técnica simple para proyección de riesgos. Consiste en 4 columnas: descripción del riesgo, categoría (ej: riesgo empresarial), probabilidad de ocurrencia (estimada por los miembros del equipo) y valor de impacto (nro. que identifica el nivel de gravedad: 1-catastrófico, 2-crítico, 3-marginal, 4-despreciable). Una vez identificados los posibles riesgos, la lista debe ordenarse según probabilidad de ocurrencia e impacto en forma descendente, a continuación se define una línea de corte y para todo riesgo que se encuentre por encima de dicha línea se debe definir un plan de mitigación, monitoreo y manejo de riesgo.

La probabilidad del riesgo puede determinarse al hacer estimaciones individuales y luego desarrollar un solo valor de consenso.



Administración del Riesgo

Riesgo de Software

Por ejemplo, suponga que una alta rotación de personal se observa como un riesgo de proyecto. Para mitigar este riesgo se desarrollará una estrategia a fin de reducir la rotación. Entre los posibles pasos por tomar están:

- Reunirse con el personal actual para determinar las causas de la rotación (por ejemplo, pobres condiciones laborales, salario bajo, mercado laboral competitivo).
- Mitigar aquellas causas que están bajo su control antes de comenzar el proyecto.
- Una vez iniciado el proyecto, suponer que la rotación ocurrirá y desarrollar técnicas para asegurar la continuidad cuando el personal se vaya.
- Organizar equipos de trabajo de modo que la información acerca de cada actividad de desarrollo se disperse ampliamente.
- Definir estándares de producto operativo y establecer mecanismos para asegurar que todos los modelos y documentos se desarrollen en forma oportuna.
- Realizar revisiones de pares de todo el trabajo (de modo que más de una persona “se ponga al día”).
- Asignar un miembro de personal de respaldo para cada técnico crítico.



Administración del Riesgo

Riesgo de Software

Conforme avanza el proyecto, comienzan las actividades de *monitoreo de riesgos*. El gerente de proyecto monitorea factores que pueden proporcionar un indicio de si el riesgo se vuelve más o menos probable. En el caso de alta rotación de personal se monitorean: la actitud general de los miembros del equipo con base en las presiones del proyecto, el grado en el que el equipo cuaja, relaciones interpersonales entre miembros del equipo, potenciales problemas con la compensación y beneficios, y la disponibilidad de empleos dentro de la compañía y fuera de ella

El *manejo del riesgo* y la *planificación de contingencia* suponen que los esfuerzos de mitigación fracasaron y que el riesgo se convirtió en realidad. Continuando con el ejemplo, el proyecto ya está en marcha y algunas personas anuncian que renunciarán al mismo. Si se siguió la estrategia de mitigación, está disponible el respaldo, la información se documentó y el conocimiento se dispersó a través del equipo.





Roles

Importancia de los roles en los proyectos de softwares

El desarrollo de software es inherentemente un esfuerzo de equipo que requiere la coordinación entre especialistas con habilidades y responsabilidades complementarias.

Una definición clara de los roles ayuda a:

- Establecer la responsabilidad
- Mejorar la comunicación
- Asegurar que todos los aspectos del desarrollo estén cubiertos
- Reducir la superposición y la confusión

Cada rol aporta una experiencia y una perspectiva única para crear un producto cohesivo y exitoso.





Roles

Rol del Desarrollador

Responsabilidades principales

- Escribir código limpio y fácil de mantener
- Implementar funciones de acuerdo con las especificaciones
- Traducir diseños técnicos en software funcional
- Realizar pruebas unitarias en los componentes desarrollados

Habilidades requeridas

- Dominio de los lenguajes de programación
- Habilidades para resolver problemas
- Comprensión de la arquitectura del software
- Control de versiones y documentación

Actividades diarias

- Codificación y depuración
- Revisiones de código
- Discusiones técnicas
- Aprender nuevas tecnologías





Roles

Rol del Evaluador

Responsabilidades principales

Los especialistas en control de calidad se aseguran de que el software funcione según lo previsto y cumpla con los estándares de calidad mediante:

- Diseño y ejecución de casos de prueba
- Identificación y documentación de errores
- Verificación de correcciones y realización de pruebas de regresión (pruebas para certificar que los cambios realizados no incorporaron nuevas fallas, es decir, que el software no sufrió regresiones).
- Garantizar que el software cumpla con los requisitos y los estándares de calidad

Tipos de pruebas

- Pruebas funcionales
- Pruebas de rendimiento
- Pruebas de seguridad
- Pruebas de aceptación del usuario





Roles

Rol del Analista Funcional

Recopilar las necesidades del usuario

Realiza entrevistas y talleres con las partes interesadas para comprender sus necesidades y expectativas.

Traducir a requisitos

Convierte las necesidades empresariales en requisitos técnicos detallados que los desarrolladores pueden implementar.

Comunicación puente

Actúa como intermediario entre el equipo técnico y las partes interesadas del negocio, asegurando la comprensión mutua.

Documentar especificaciones

Crea documentación clara y detallada de los requisitos del sistema y las reglas de negocio.

El analista funcional es crucial para garantizar que el producto final aborde las necesidades reales del negocio.



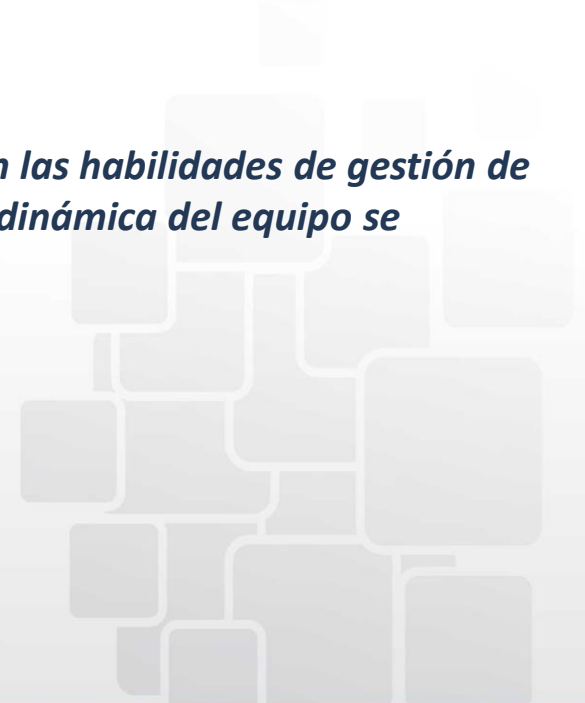
Roles

Rol del Líder de Proyecto

Responsabilidades claves

- Coordinar los esfuerzos del equipo y asignar recursos
- Planificar el cronograma y los hitos del proyecto
- Monitorear el progreso en función de los objetivos
- Identificar y mitigar los riesgos
- Informar el estado a las partes interesadas
- Resolver conflictos y eliminar obstáculos

Los líderes de proyecto eficaces equilibran la comprensión técnica con las habilidades de gestión de personas, asegurando que tanto los resultados del proyecto como la dinámica del equipo se mantengan saludables.





Roles

Rol del Usuario/Cliente

Definición de necesidades

- Articula los problemas y oportunidades de negocio que el software debe abordar.
- Proporciona contexto sobre procesos, limitaciones y objetivos.

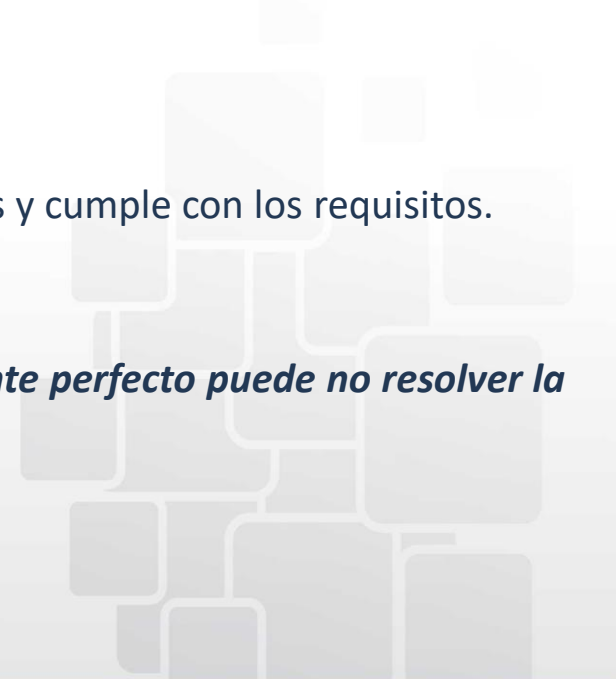
Proporcionar retroalimentación

- Revisa los prototipos y las primeras versiones para garantizar la alineación con las expectativas.
- Sugiere ajustes y mejoras durante todo el desarrollo.

Validación de soluciones

- Confirma si el producto entregado resuelve los problemas previstos y cumple con los requisitos.
- Pruebas de aceptación final y aprobación.

Sin la participación activa del cliente, incluso un software técnicamente perfecto puede no resolver la necesidad real del negocio.





Roles

Roles en Metodologías Ágiles

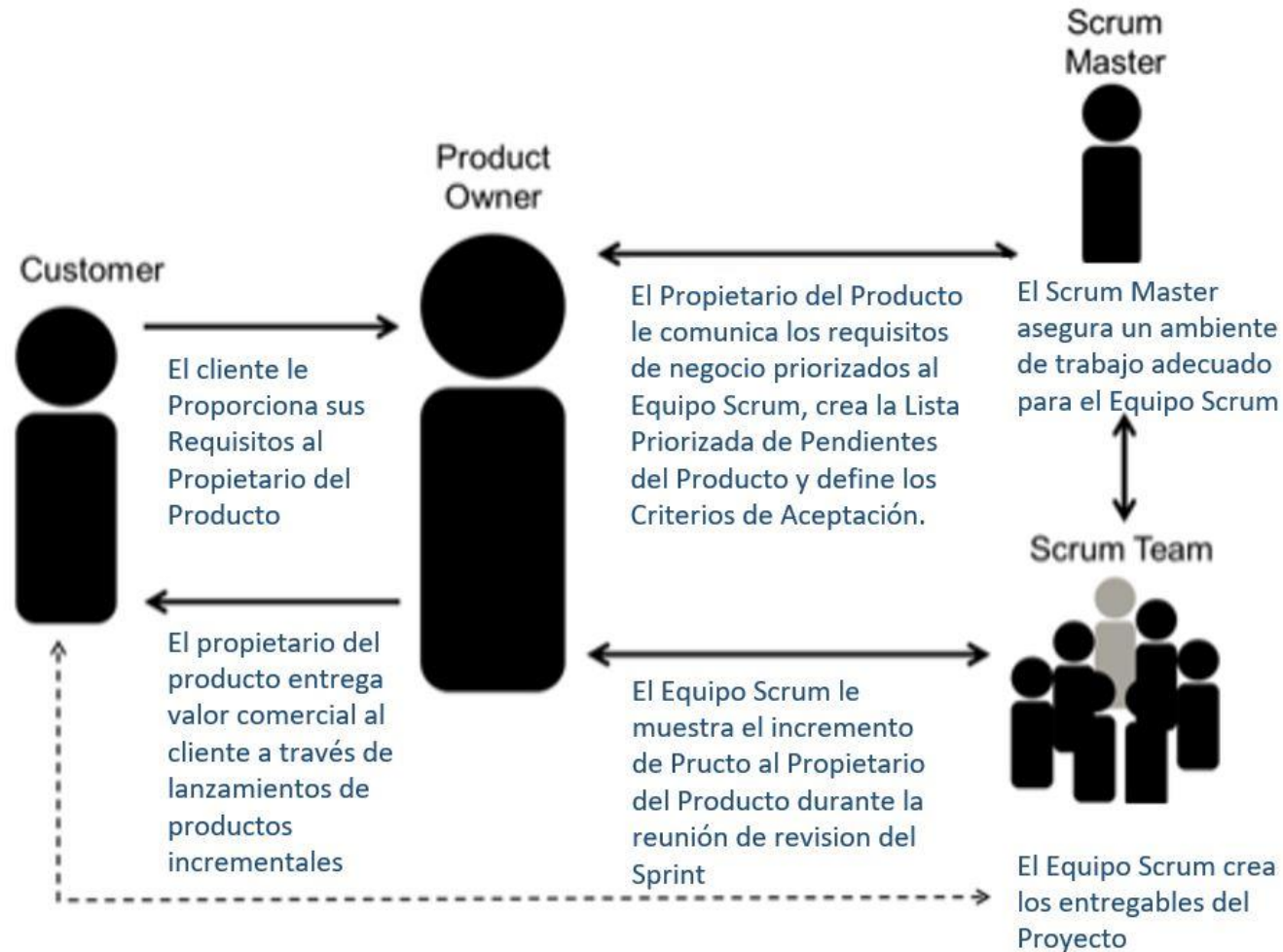
Rol	Función Principal	Responsabilidades Claves
Product Owner	Representa los intereses comerciales	Prioriza las funciones, gestiona el backlog del producto (lista de tareas pendientes), maximiza el valor
Scrum Master	Facilitador de Proceso	Elimina los impedimentos, garantiza el cumplimiento de las prácticas ágiles, capacita al equipo
Equipo Ágil	Constructor del Producto	Colaboración interfuncional, progreso diario, entrega continua

Las metodologías ágiles enfatizan la flexibilidad, la colaboración y la responsabilidad compartida en lugar de las definiciones de roles rígidas.



Roles

Roles en Metodologías Ágiles





Requerimientos

Un requisito es una condición o capacidad específica que debe cumplir un sistema para satisfacer un contrato, estándar, especificación u otro documento impuesto formalmente.

"Los requisitos son la base sobre la que se construye el software. Si la base es débil, el software fallará, independientemente de lo bien que esté diseñado o construido".

Los requisitos definen tanto lo que el sistema debe hacer (funcionalidad) como la forma en que debe funcionar (atributos de calidad).

Requerimientos funcionales

Definen lo que debe hacer el sistema:

- Características y capacidades
- Comportamientos específicos
- Procesos de entrada/salida

Requerimientos no funcionales

Definen cómo debe funcionar el sistema, fijan restricciones sobre los requisitos funcionales:

- Criterios de rendimiento
- Medidas de seguridad
- Estándares de usabilidad
- Métricas de confiabilidad





Requerimientos

Requerimientos Funcionales	Requerimientos No Funcionales
El sistema debe permitir a los usuarios registrarse con correo electrónico y contraseña	El registro de usuario debe completarse en 2 segundos
El sistema debe enviar correos electrónicos de confirmación después del registro	El sistema debe ser compatible con todos los dispositivos móviles
Los usuarios deben poder restablecer sus contraseñas	El proceso de restablecimiento de contraseña debe cumplir con las regulaciones del RGPD (Reglamento General de Protección de Datos)
El sistema debe generar informes de asistencia mensuales	Los informes deben ser accesibles para usuarios con discapacidades visuales
El sistema permitirá a los profesores marcar a los estudiantes como presentes, ausentes o tardíos con una sola acción	Se debe poder acceder al sistema a través de dispositivos móviles con pantallas de hasta 4 pulgadas



Requerimientos

Errores comunes al momento de relevar requerimientos

Ambigüedad – Lenguaje vago

Los requisitos con múltiples interpretaciones posibles conducen a malentendidos.

Ejemplo: "El sistema debe responder rápidamente" (¿Qué significa "rápidamente"? ¿1 segundo? ¿5 segundos?).

Suposiciones no validadas

Dar por sentada información que no ha sido confirmada.

Ejemplo: Asumir que todos los usuarios tienen conexiones a Internet de alta velocidad (reducir tráfico de datos) o que tendrán Excel disponible para visualizar los informes (proporcionar otros formatos ej. pdf).

Cambios no documentados

Los requisitos evolucionan, pero los cambios no se rastrean adecuadamente.

Ejemplo: Acuerdos verbales para agregar funciones sin actualizar la documentación

Combinar Requisitos

Interpretar múltiples requisitos como un único requerimiento.

Ejemplo: "La página de inicio de sesión debe usar tokens JWT y verse profesional". Plantear 2 requisitos por separado: "El sistema debe implementar JWT para la autenticación" y "La interfaz de usuario debe seguir las pautas de marca de la empresa".



Requerimientos

De los objetivos se desprenden los requisitos

A la hora de definir requisitos, es indispensable consultar al cliente por sus necesidades, las características de sus problemas, el tipo de servicio que pretende y las utilidades que precisa. Es vital para el proyecto, acompañar al usuario en esta etapa de exploración y búsqueda, indagando qué síntomas percibe y analizando las posibles causas.

La identificación de requisitos es una responsabilidad compartida entre el profesional y el usuario. Mientras el usuario indaga en sus propias percepciones, el analista, define características vinculadas con los procesos de desarrollo. En tanto su objetivo se centra en encontrar el problema real del cliente, su análisis trasciende las limitaciones de lo que éste manifiesta (*pensamiento sistémico*).

Los requisitos claros y bien definidos proporcionan la base para un desarrollo de software exitoso, reduciendo la costosa reelaboración y garantizando la alineación con las necesidades del negocio.





Requerimientos

Técnicas de relevamiento

Entrevistas con las partes interesadas

- Conversaciones individuales o grupales con usuarios y tomadores de decisiones para comprender las necesidades y expectativas.

Cuestionarios

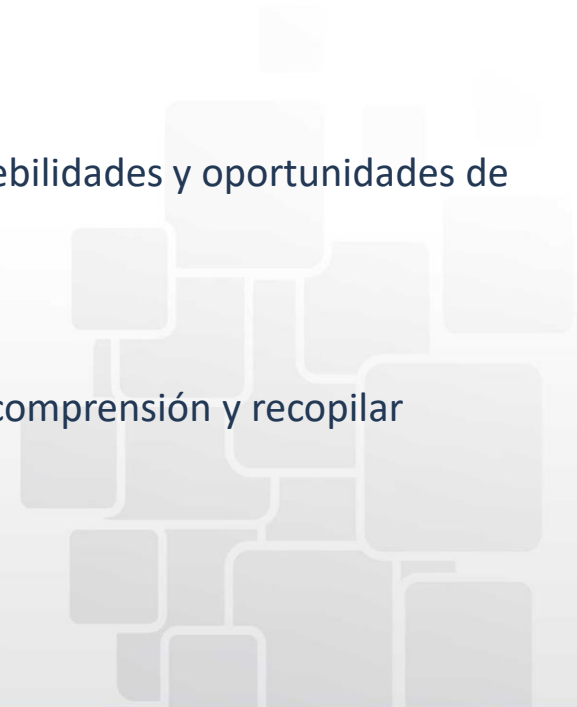
- Formularios estructurados para recopilar información coherente de múltiples partes interesadas, especialmente útiles para equipos distribuidos.

Análisis del sistema actual

- Examinar las soluciones actuales para comprender las fortalezas, debilidades y oportunidades de mejora.

Prototipos

- Representaciones visuales de la solución propuesta para validar la comprensión y recopilar comentarios de forma temprana.



Muchas gracias por su atención.

